Chapter 1: Basic Python Programming

Fundamentals of Python: Data Structures, Second Edition

Learning Objectives (1 of 2)

- Write a simple Python program using its basic structure
- Perform simple input and output operations
- Perform operations with numbers such as arithmetic and comparisons
- Perform operations with Boolean values
- Implement an algorithm using the basic constructs of sequences of statements, selection statements, and loops
- Define functions to structure code

Learning Objectives (2 of 2)

- Use built-in data structures such as strings, files, lists, tuples, and dictionaries
- Define classes to represent new types of objects
- Structure programs in terms of cooperating functions, data structures, classes, and modules

Basic Program Elements

- Python has a vast array of features and constructs
  - Basic program elements are quite simple

Programs and Modules

- Python programs consist of one or more modules
- Module: a file of Python code
  - Can include statements, function definitions, and class definitions
- Script: a short Python program
  - Can be contained in one module
- Longer programs typically include
  - One main module and one or more supporting modules
- Main module
  - Contains starting point of program execution
- Supporting modules
  - Contain function and class definitions

An Example Python Program: Guessing a Number (1 of 2)

```
>"""
Author: Ken Lambert
Plays a game of guess the number with the user.
"""

import random

def main():
    """Inputs the bounds of the range of numbers
    and lets the user guess the computer's number until
    the guess is correct."""
    smaller = int(input("Enter the smaller number: "))
    larger = int(input("Enter the larger number: "))
    myNumber = random.randint(smaller, larger)
    count = 0
    while True:
        count += 1
        userNumber = int(input("Enter your guess: "))
        if userNumber < myNumber:
            print("Too small")
        elif userNumber > myNumber:
            print("Too large")
        else:
            print("You've got it in", count, "tries!")
            break

if __name__ == "__main__":
    main()
```

An Example Python Program: Guessing a Number (2 of 2)

- Here is a trace of a user's interaction with the program:
  - Enter the smaller number: 1
  - Enter the larger number: 32
  - Enter your guess: 16
  - Too small
  - Enter your guess: 24
  - Too large
  - Enter your guess: 20
  - You've got it in 3 tries!

Editing, Compiling, and Running Python Programs (1 of 2)

- To run the program contained in the file numberguess.py, enter the following command in most terminal windows:
  - **`python3 numberguess.py`**
- To create or edit a Python module, use Python's IDLE:
  - Enter the **`idle`** or **`idle3`** command at a terminal prompt or click icon
  - You can also launch IDLE by
    - Double-clicking on a Python source code file
    - Right-clicking on the file and selecting Open or Edit with IDLE
- IDLE gives you a shell window for interactively running Python expressions and statements:
  - IDLE also formats your code and color-codes it

Editing, Compiling, and Running Python Programs (2 of 2)

- To run a program within IDLE
  - Move the cursor into the editor window and press the F5 key
- Python compiles the code in the editor window and runs it in the shell window
- If a Python program appears to hand or not quit normally,
  - Exit by pressing Ctrl+C or close the shell window

Program Contents

- Program comment
  - Text ignored by the Python compiler
  - Valuable to the reader as documentation
- End-of-line comment
  - Begins with a # symbol and extends to the end of the current line
- Multiline comment
  - A string enclosed in triple single quotes or triple double quotes
  - Called *docstrings* to indicate that they can document major constructs within a program

Lexical Elements

- Lexical elements
  - Types of words or symbols used to construct sentences
- Lexical items also include
  - Identifiers (names)
  - Literals (numbers, strings, and other built-in data structures)
  - Operators
  - Delimiters (quotation marks, commas, parentheses, square brackets, and braces)

Spelling and Naming Conventions (1 of 2)

- Python keywords and names ARE case-sensitive:
  - Keywords are spelled in lowercase letters and color-coded in orange
  - All names (other than those of built-in functions) are color-coded in black
- A name can begin with a letter or an underscore (_)
  - Followed by an number of letters, underscores, or digits
- In this book
  - Names of modules, variables, functions, and methods are spelled in lowercase letters
  - Names of classes follow the same conventions but begin with a capital letter
  - When a variable names a constant, all letters are uppercase and an underscore separates embedded words

Spelling and Naming Conventions (2 of 2)

- Table 1.1 Examples of Python Naming Conventions

Type of Name	Examples
Variable	`salary, hoursWorked, isAbsent`
Constant	`ABSOLUTE_ZERO, INTEREST_RATE`
Function or method	`printResults, cubeRoot, input`
Class	`BankAccount, SortedSet`

Syntactic Elements

- Syntactic elements
  - Types of sentences (expressions, statements, definitions, and other constructs) composed from lexical elements
- Python uses white space to mark the syntax of many types of sentences:
  - Indentation and line breaks
- This book uses an indentation width of four spaces in all Python code

Literals

- Numbers are written as they are in other programming languages
- Boolean values **`True`** and **`False`** are keywords
- Some data structures also have literals
  - Such as string, tuples, lists, and dictionaries

String Literals (1 of 2)

- Enclose strings in
  - Single quotes
  - Double quotes
  - Sets of three double quotes or three single quotes
- Character values are single-character strings
- The \ character is used to escape nongraphic characters such as
  - Newline (\n) and tab (\t) or the \ character itself

String Literals (2 of 2)

- Example:
  - `print("Using double quotes")`
  - `print('Using single quotes')`
  - `print("Mentioning the word 'Python' by quoting it")`
  - `print("Embedding a\nline break with \n")`
  - `print("""Embedding a`
  - `line break with triple quotes""")`
- Output:

```
Using double quotes
Using single quotes
Mentioning the word 'Python' by quoting it
Embedding a
line break with \n
Embedding a
line break with triple quotes
```

Operators and Expressions (1 of 2)

- Arithmetic expressions use
  - `+`, `-`, `*`, and `%`
- The `/` operator produces a floating-point result with any numeric operands
- The `//` operator produces an integer quotient
- The `+` operator means concatenation when used with collections
- The `**` operator is used for exponentiation

Operators and Expressions (2 of 2)

- Comparison operators work with numbers and strings
  - `<`, `<=`, `>`, `>=`, `==`, and `!=`
- The `==` operator compares the internal contents of data structures for structural equivalence
- The is operator compares two values for object identity
- Comparisons return **`True`** or **`False`**
- Logical operators treat several values, such as 0, None, the empty string, and the empty list, as **`False`**:
  - Most other Python values count as **`True`**

Function Calls

- Functions are called with
  - Function's name followed by a list of arguments
- Example:
  - `min (5,2)    #Returns 2`

The `print` Function

- **`print`**
  - Standard output function that displays arguments on the console
- Python automatically runs the **`str`** function on each argument to obtain its string representation:
  - Separates each string with a space before output
- By default
  - **`print`** terminates its output with a newline

The `input` Function

- **`input`**
  - Waits for the user to enter text
- When user presses the Enter key,
  - The function returns a string containing characters entered

Type Conversion Functions and Mixed-Mode Operations

- You can use some data type names as type conversion functions
- Example:
  - When the user enters a number at a keyboard, the **`input`** function returns a string of digits, not a numeric value
  - Program must convert the string to an **`int`** or **`float`** before numeric processing
- This code inputs the radius of a circle, converts this string to a float, and computes and outputs the circle's area:

```
radius = float(input("Radius: "))

print("The area is", 3.14 * radius ** 2)
```

Optional and Keyword Function Arguments

- Functions may allow optional arguments
  - Can be named with keywords when the function is called
- Example:
  - To prevent the print function to output a newline after its arguments are displayed, give the optional argument end a value of an empty string

```
print("The cursor will stay on this line, at the end", end = "")
```

- Required arguments have no default values
  - Optional arguments have default values
- Number of arguments passed to a function must be at least the same number as its required arguments

Variables and Assignment Statements

- Python variables are introduced with an assignment statement:
  - `PI = 3.1416`
- Syntax:
  - `<identifier> = <expression>`
- Several variables can be introduced in the same statement:
  - `minValue, maxValue = 1, 100`
- Assignment statements must appear on a single line of code
  - Unless the line is broken after a comma, parenthesis, curly brace, or square bracket
  - Can also end it with the escape symbol \

Python Data Typing

- Any variable can name a value of any type
- Variables are not declared to have a type:
  - They are simply assigned a value
- Data type names almost never appear in Python programs:
  - However, all values or objects have types

`import` Statements

- Import statement makes visible to a program the identifiers from another module
- Several ways to express an import statement:
  - Simplest is to import the module name:

```
import math
```

  - Another option brings in a name itself:

```
from math import sqrt
print (sqrt (2))
```

- You can import several individual names by listing them:

```
from math import pi, sqrt
print (sqrt (2) * pi)
```

Getting Help on Program Components

- To access help,
  - Enter the function call **`help (<component>)`** at the shell prompt
- For example,
  - **`help (abs)`** and **`help (math.sqrt)`** display documentation for the **`abs`** and **`math.sqrt`** functions
- Calls of **`dir(int)`** and **`dir(math)`** list all the operations in the **`int`** type and **`math`** module:
  - You can then run help to get help on one of these operations

Control Statements

- A sequence of statements is a set of statements written one after the other
- Each statement in a sequence must begin in the same column

Conditional Statements (1 of 2)

- Keywords **`if`**, **`elif`**, and **`else`** are significant
  - As is the colon character and indentation
- Syntax of the one-way **`if`** statement:

```
if <Boolean expression>:
    <sequence of statements>
```

- Syntax of the two-way if statement:

```
if <Boolean expression>:
    <sequence of statements>
else:
    <sequence of statements>
```

Conditional Statements (2 of 2)

- Syntax of the multiway if statement:

```
if <Boolean expression>:
    <sequence of statements>
elif <Boolean expression>:
    <sequence of statements>
else:
    <sequence of statements>
```

- Example:

```
if x > y:
    print ("x is greater than y")
elif x < y:
    print ("x is less than y")
else:
    print ("x is equal to y")
```

Using `if _name_ == "_main_"`

- The **`numberguess`** program includes the definition of a main function and the following **`if`** statement:

```
if _name_ == "_main_":
    main ()
```

- Purpose of this **`if`** statement is to allow the programmer either to run the module as a standalone program or to import it from the shell or other module

Loop Statements

- Syntax of a **`while`** loop statement:

```
while <Boolean expression>:
    <sequence of statements>
```

- Syntax of a **`for`** loop statement:

```
for <variable> in <iterable object>:
    <sequence of statements>
```

- Python programmers prefer a **`for`** loop to iterate over definite ranges or sequences of values:
  - They use a **`while`** loop when the continuation condition is an arbitrary Boolean expression

Strings and Their Operations

- A Python string is a compound object that includes other objects
  - Namely, its characters
  - Each character in a Python string is a single-character string
- Python's string type, named **`str`**
  - Includes a large set of operations

Operators

- Comparison operators:
  - When strings are compared, the pairs of characters at each position in two strings are compared
- The + operator builds and returns a new string that contains the characters of the two operands
- The subscript operator expects an integer in the range from 0 to the length of the string minus 1:

```
"greater"[0]   # Returns 'g'
```

- Slice operator is a variation of the subscript:

```
"greater"[:] # Returns "greater"
"greater"[2:]     # Returns "eater"
"greater"[:2]     # Returns "gr"
"greater"[2:5]    # Returns "eat"
```

Formatting Strings for Output (1 of 2)

- Python includes a general formatting mechanism that allows the programmer to specify field widths for different types of data
- Example of how to right justify and left justify the string "four" within a field width of 6:

```
>>>"%6s" % "four"   # Right justify
'   four'
>>>"%-6s" % "four"  # Left justify
'four   '
```

Formatting Strings for Output (2 of 2)

- The format information for a data value of type float has the form:

  %<field width>.<precision>f

- This example shows the output of a floating-point number without, and then with, a format string:

```
>>>salary = 100.00
>>>print("Your salary is $" + str(salary))
Your salary is $100.0
>>>print("Your salary is $%0.2f" % salary)
Your salary is $100.00
```

Object and Method Calls

- Syntax of method call:
  - `<object>.<method name>(<list of arguments>)`
- Examples of method calls on strings:

```
"greater".isupper()              # Returns False
"greater".upper()                # Returns "GREATER"
"greater".startswith("great") # Returns True
```

- If you try to run a method that an object does not recognize,
  - Python raises an exception and halts the program
- To discover the set of methods that an object recognizes,
  - Run Python's `dir` function, in the Python shell, with the object's type as an argument

Built-In Python Collections and Their Operations

- Modern programming languages include several types of collections, such as lists
  - That allow the programmer to organize and manipulate several data values at once
- This section explores the built-in collections in Python

Lists (1 of 2)

- List
  - A sequence of zero or more Python objects
  - Commonly called items
  - Has a literal representation
  - Uses square brackets to enclose items separated by commas
- Examples:

```
[]                              # An empty list
["greater"]                     # A list of one string
["greater", "less"]             # A list of two strings
["greater", "less", 10]         # A list of two strings and an int
["greater", ["less", 10]]       # A list with a nested list
```

Lists (2 of 2)

- Most commonly used list mutator methods:
  - **append, insert, pop, remove, and sort**
- Examples:

```
testList = []                  # testList is []
testList.append(34)            # testList is [34]
testList.append(22)            # testList is [34, 22]
testList.sort()                # testList is [22, 34]
testList.pop()                 # Returns 22; testList is [34]
testList.insert(0, 22)         # testList is [22, 34]
testList.insert(1, 55)         # testList is [22, 55, 34]
testList.pop(1)              # Returns 55; testList is [22, 34]
testList.remove(22)            # testList is [34]
testList.remove(55)            # raises ValueError
```

Tuples

- Tuple
  - An immutable sequence of items
  - Tuple literals enclose items in parentheses
  - Essentially like a list without mutator methods
- A tuple with one item must still include a comma:

```
>>> (34)
34

>>> (34,)
(34)
```

Loops over Sequences

- For loop is used to iterate over items in a sequence:

```
testList = [67, 100, 22]
for item in testList:
      print(item)
```

- This is equivalent to but simpler than an index-based loop over the list:

```
testList = [67, 100, 22]
for index in range(len(testList)):
      print(testList[index])
```

Dictionaries

- Dictionary
  - Contains zero or more entries
  - Each entry associates a unique key with a value
    - Keys are typically string or integers
    - Values are any Python objects
- Examples:

```
{}                                          # An empty dictionary
{"name":"Ken"}                               # One entry
{"name":"Ken", "age":67}                     # Two entries
{"hobbies":["reading", "running"]}           # One entry, value
                                               is a list
```

```
>>> for key in {"name":"Ken", "age":67}:
    print(key)
name
age
```

Searching for a Value

- The programmer can search strings, lists, tuples, or dictionaries for a given value by running the `in` operator
  - With the value and the collection
  - This operator returns `True` or `False`
- For dictionaries
  - The methods `get` and `pop` can take two arguments: a key and a default value
  - A failed search returns the default value

Pattern Matching with Collections

- The subscript can be used to access items within lists, tuples, and dictionaries
  - It is often more convenient to access several items at once by means of pattern matching
- Example using the subscript operator on **`colorTuple`**:

```
rgbTuple = colorTuple[0]
hexString = colorTuple[1]
r = rgbTuple[0]
g = rgbTuple[1]
b = rgbTuple[2]
```

Creating New Functions

- Python includes built-in functions and allows the programmer to create new functions:
  - New functions can utilize recursion
  - Can receive and return functions as data

Function Definitions

- Syntax of a Python function definition:

```
def <function name>(<list of parameters>):
    <sequence of statements>
```

- Rules and conventions for spelling function names and parameter names are the same as for variable names
- Example of a definition of a simple function to compute and return the square of a number:

```
def square(n):
    """Returns the square of n."""
    result = n ** 2
    return result
```

Recursive Functions (1 of 2)

- Recursive function
  - A function that calls itself
- To prevent a function from repeating itself indefinitely,
  - It must contain at least one section statement that examines a condition called a *base case* to determine whether to stop or to continue with a recursive step
- Example of a definition of a function **displayRange** that prints the numbers from a lower bound to an upper bound:

```
def displayRange(lower, upper):
    """Outputs the numbers from lower to upper."""
    while lower <= upper:
        print(lower)
        lower = lower + 1
```

Recursive Functions (2 of 2)

- How would you go about converting the previous function to a recursive one? Note two important facts:
  - The loop's body continues execution while **`lower <= upper`**
  - When the function executes, **`lower`** is incremented by 1 but **`upper`** never changes
- The loop is replaced with an **`if`** statement, and the assignment statement is replaced with a recursive call of the function:

```
def displayRange(lower, upper):
    """Outputs the numbers from lower to upper."""
    if lower <= upper:
        print(lower)
        displayRange(lower + 1, upper)
```

Nested Function Definitions

- Definitions of other functions may be nested within a function's sequence of statements:

```
# First definition
def factorial(n):
    """Returns the factorial of n."""

    def recurse(n, product):
        """Helper function to compute factorial."""
        if n == 1: return product
        else: return recurse(n - 1, n * product)

    return recurse(n, 1)

# Second definition
def factorial(n, product = 1):
    """Returns the factorial of n."""
    if n == 1: return product
    else: return factorial(n - 1, n * product)
```

Higher-Order Functions (1 of 2)

- Higher-order function
  - A function that receives another function as an argument and applies it in some way
- Python includes two built-in higher-order functions useful for processing iterable objects:
  - **`map`** and **`filter`**

```
map(str, oldList)
```

- Creates the iterable object containing the strings, and the code

```
newList = list(map(str, oldList))
```

- Creates a new list from that object

Higher-Order Functions (2 of 2)

- The `filter` function returns an iterable object in which each item is passed to the Boolean function:
  - Essentially keeps the items that pass a test in an iterable object

```
filter(isPositive, oldList)
```

  - Creates the iterable object containing the non-zero grades, and the code

```
newList = list(filter(isPositive, oldList))
```

  - Creates a new list from that object

Creating Anonymous Functions with `lambda`

- Syntax of **`lambda`**:

```
lambda <argument list> : <expression>
```

- The code:

```
newList = list(filter(lambda number: number > 0, oldList))
```

- Uses an anonymous Boolean function to drop the zero grades from the list of grades

Catching Exceptions

- Python includes a **`try-except`** statement that
  - Allows a program to trap or catch exceptions and perform the appropriate recovery operations
- Syntax of simplest form of this statement:

```
try:
   <statements>
except <exception type>:
   <statements>
```

- In general, you should try to include the exception type that matches the type of exception expected under the circumstances:
  - If no such type exists, the general **`Exception`** type will match any exception

Files and Their Operations

- Python provides great support for managing and processing several types of files
- This section examines some manipulations of text files and object files

Text File Output (1 of 2)

- When data are treated as integers or floating-point numbers,
  - They must be separated by whitespace characters (spaces, tabs, and newlines)
- A text file containing six floating-point numbers might look like:

```
34.6 22.33 66.75
77.12 21.44 99.01
```

- All data output to or input from a text file must be strings:
  - Numbers must be converted to strings before output
  - These strings must be converted back to numbers after input
- You can output data to a text file using a file object:
  - Python's open function, which expects a file pathname and a mode string as arguments, opens a connection to the file on disk and returns a file object

Text File Output (2 of 2)

- String data are written to a file using the method **`write`** with the file object:
  - The write method expects a single string argument
- If you want the output text to end with a newline,
  - You must include the escape character `\n` in the string
- The next statement writes two lines of text to the file:
  - **`>>> f.write("First line.\nSecond line.\n")`**
- When all the outputs are finished, the file should be closed using the method **`close`**, as follows:
  - **`>>> f.close()`**

Writing Numbers to a Text File

- Integers or floating-point numbers must first be converted to strings before being written to an output file
- Values of most data types can be converted to strings by using the **`str`** function
- Resulting strings are written to a file with a space or a newline as a separator character

Reading Text from a Text File (1 of 2)

- Similar to opening a file for output, the
  - Only thing that changes is the mode string, which, in the case of opening a file for input, is ‘r’
- Code for opening myfile.txt for input:
  - `>>> f = open("myfile.txt",'r')`
- If the file contains multiple lines of text, the newline character will be embedded in this string

Reading Text from a Text File (2 of 2)

- Using the method **`read`**:

```
>>> text = f.read()
>>> text
'First line.\nSecond line.\n'
>>> print(text)
First line.
Second line.
```

- After input is finished
  - Another call to read returns an empty string
  - Indicates that the end of the file has been reached

Reading Numbers from a File

- String representations of integers and floating-point numbers can be converted to the numbers themselves
  - By using the functions **`int`** and **`float`**, respectively
- Code that opens the file of random integers written earlier, reads them, and prints their sum:

```
f = open("integers.txt",'r')
theSum = 0
for line in f:
   line = line.strip()
   number = int(line)
   theSum += number
print("The sum is", theSum)
```

Reading and Writing Objects with `pickle`

- Python includes a module that allows the programmer to save and load objects using a process called pickling
- You start by importing the **`pickle`** module:
  - Files are opened for input and output using the "**`rb`**" and "**`wb`**" flags (for byte streams) and closed in the usual manner
- To save an object, you use the function **`pickle.dump`**
- Code:

```
import pickle
lyst = [60, "A string object", 1977]
fileObj = open("items.dat", "wb")
for item in lyst:
   pickle.dump(item, fileObj)
fileObj.close()
```

Creating New Classes

- Class
  - Describes the data and the methods pertaining to a set of objects
  - Provides a blueprint for creating objects and the code to execute when methods are called on them
- Syntax of a Python class definition:

```
def <class name>(<parent class name>):

    <class variable assignments>

      <instance method definitions>
```

- See text for examples of code for a class